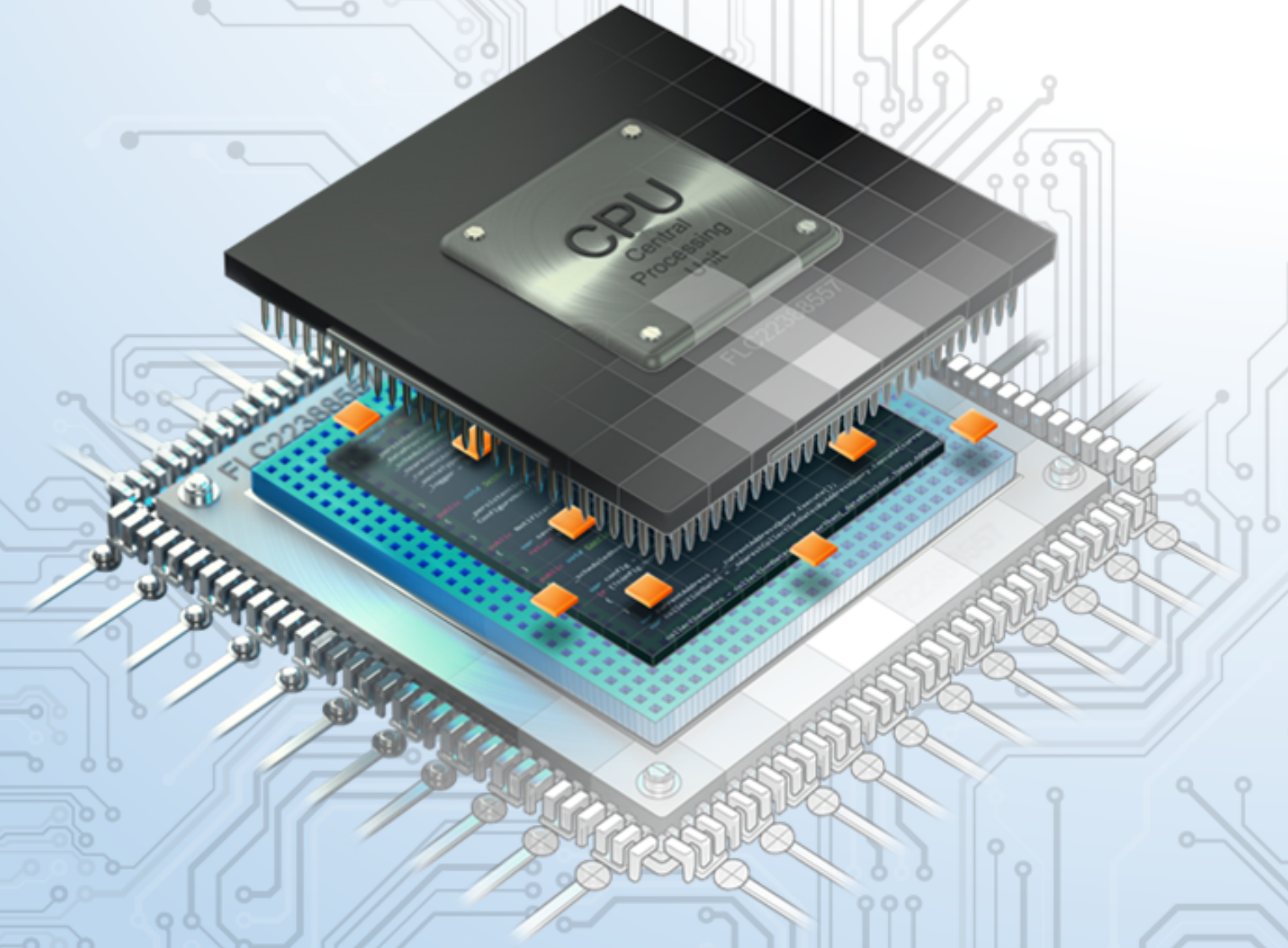
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller



Dr. Le Trong Nhan

Mục lục

Chapter 1. LED Animations **7**

1 Introduction 8

2 First project on STM32Cube 9

3 Simulation on Proteus 15

4 Exercise and Report 21

4.1 Exercise 1 21

4.2 Exercise 2 25

4.3 Exercise 3 27

4.4 Exercise 4 27

4.5 Exercise 5 30

4.6 Exercise 6 32

4.7 Exercise 7 34

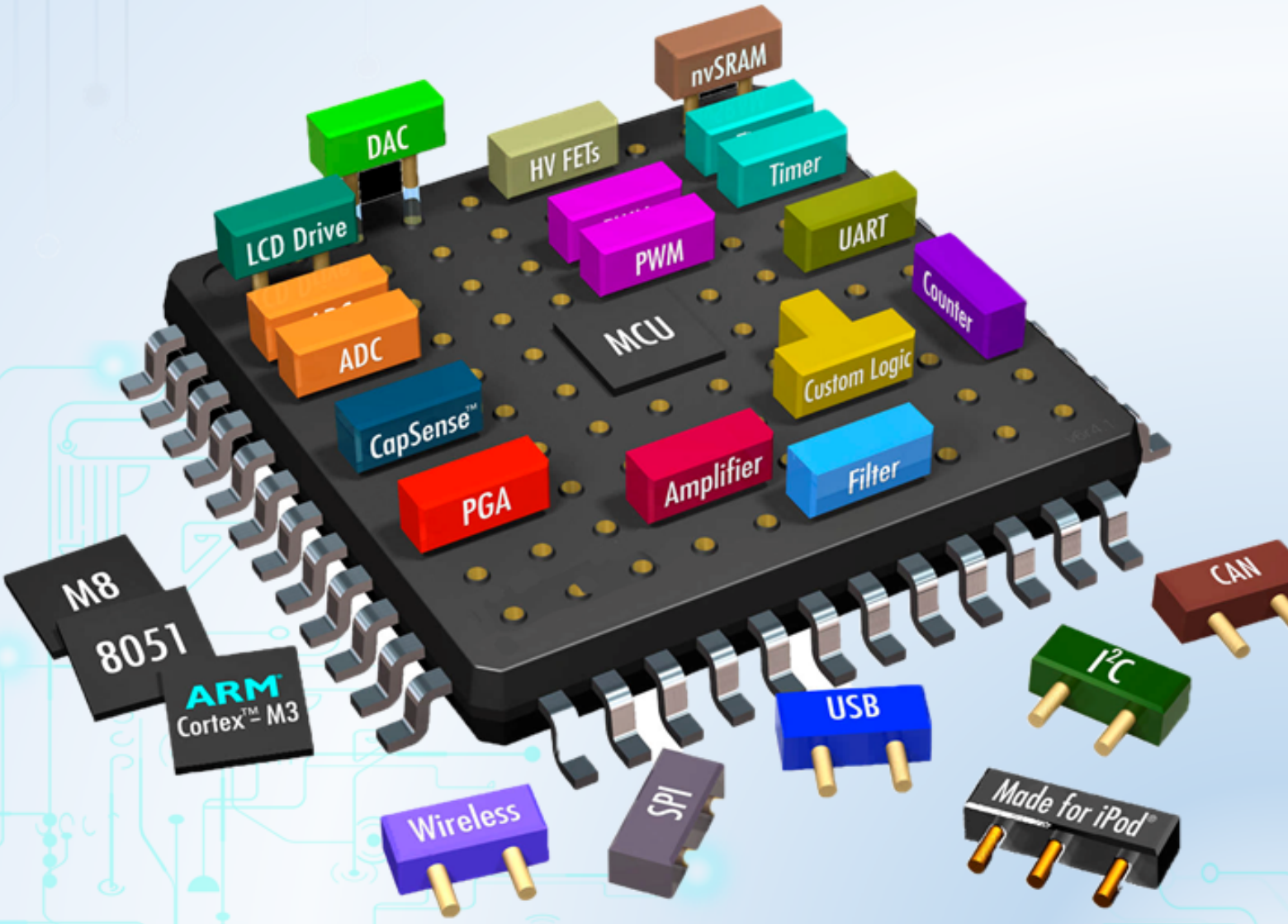
4.8 Exercise 8 35

4.9 Exercise 9 36

4.10 Exercise 10 37

CHƯƠNG 1

LED Animations



1 Introduction

In this manual, the STM32CubeIDE is used as an editor to program the ARM microcontroller. STM32CubeIDE is an advanced C/C++ development platform with peripheral configuration, code generation, code compilation, and debug features for STM32 microcontrollers and microprocessors.



Hình 1.1: *STM32Cube IDE for STM32 Programming*

The most interest of STM32CubeIDE is that after the selection of an empty STM32 MCU or MPU, or preconfigured microcontroller or microprocessor from the selection of a board, the initialization code generated automatically. At any time during the development, the user can return to the initialization and configuration of the peripherals or middleware and regenerate the initialization code with no impact on the user code. This feature can simplify the initialization process and speedup the development application running on STM32 micro-controller. The software can be downloaded from the link bellow:

https://ubc.sgp1.digitaloceanspaces.com/BKU_Softwares/STM32/stm32cubeide_1.7.0.zip

Moreover, for a hangout class, the program is firstly simulated on Proteus. Students are also supposed to download and install this software as well:

https://ubc.sgp1.digitaloceanspaces.com/BKU_Softwares/STM32/Proteus_8.10_SP0_Pro.exe

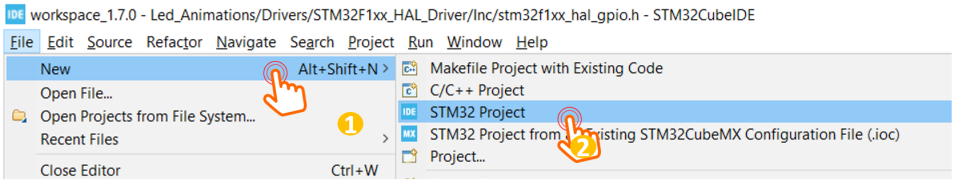
The rest of this manual consists of:

- Create a project on STM32Cube IDE
- Create a project on Proteus
- Simulate the project on Proteus

Finally, students are supposed to finish 10 different projects.

2 First project on STM32Cube

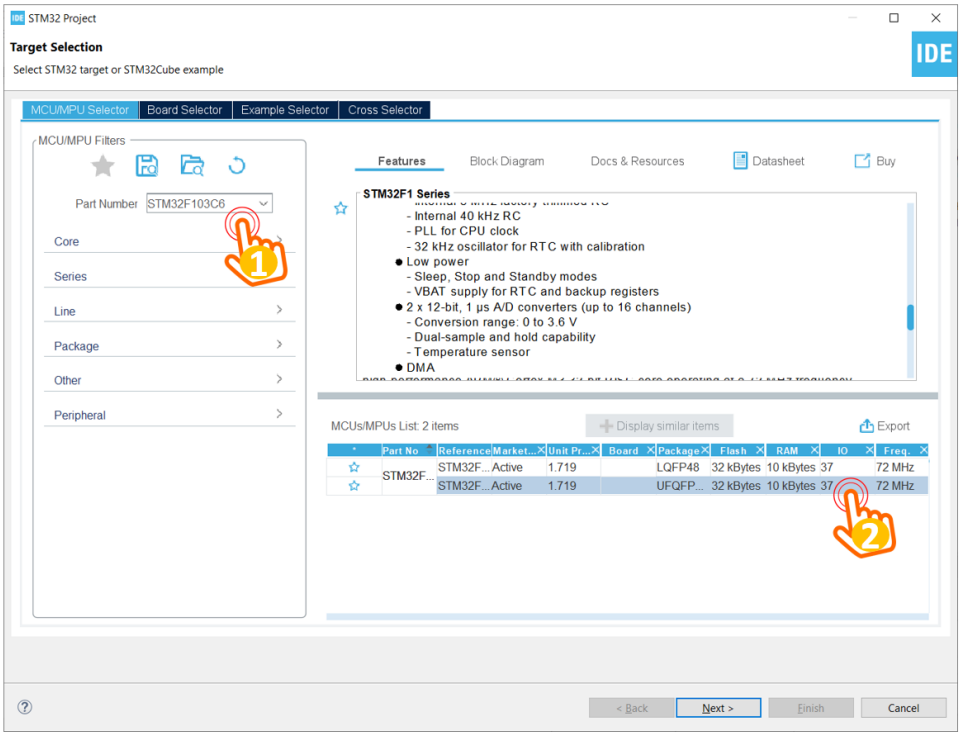
Step 1: Launch STM32CubeIDE, from the menu **File**, select **New**, then chose **STM32 Project**



Hình 1.2: Create a new project on STM32CubeIDE

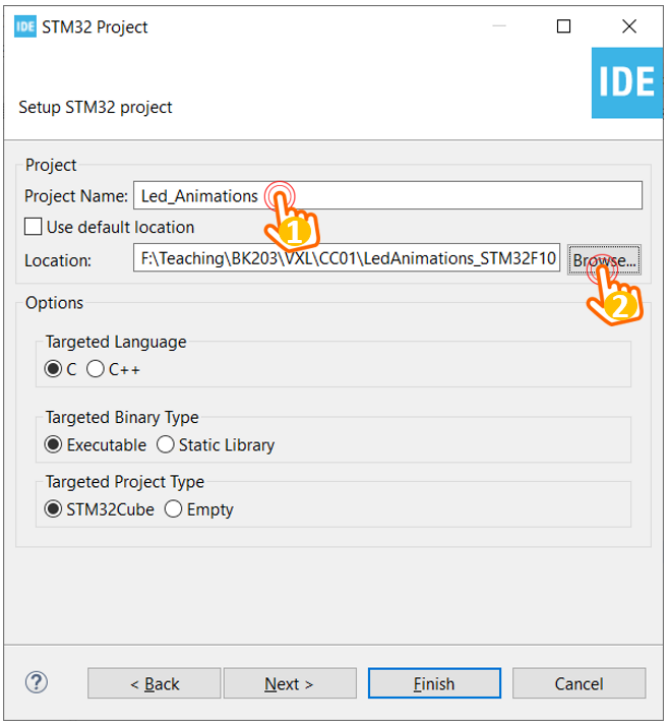
The IDE needs to download some packages, which normally takes time in this first time a project is created.

Step 2: Select the STM32F103C6 in the following dialog, then click on **Next**



Hình 1.3: Select the target device

Step 3: Provide the **Name** and the **Location** for the project.



Hình 1.4: Select the target device

It is important to notice that the **Targeted Project Type** should be **STM32Cube**. In the case this option is disable, step 1 must be repeated. The location path should not contain special characters (e.g. the space). Finally, click on the **Next** button.

Step 4: On the last dialog, just keep the default firmware version and click on **Finish** button.

Step 5: The project is created and the wizard for configuration is display. This utility from CubeIDE can simplify the configuration process for an ARM microcontroller like the STM32.

From the configuration windows, select **Pin configuration**, select the pin **PA5** and set to **GPIO Output** mode, since this pin is connected to an LED in the STM32 development kit.

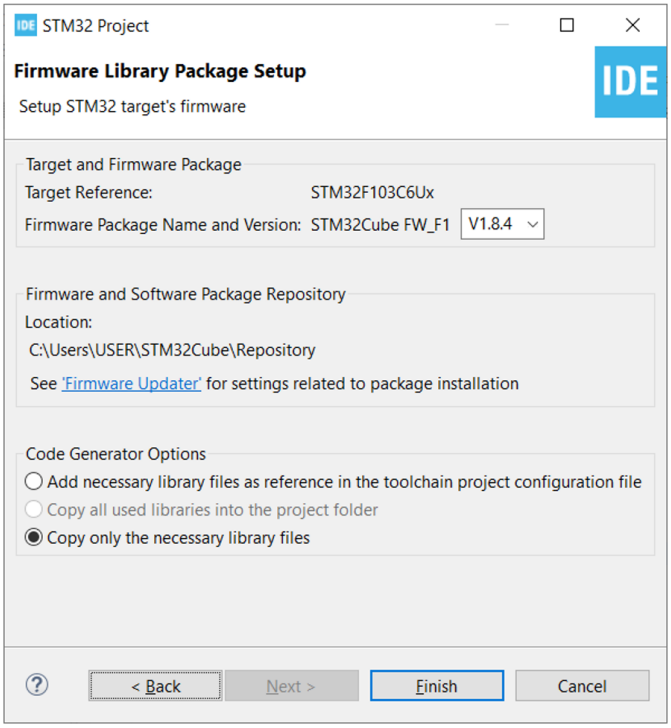
Step 6: Right click on PA5 and select **Enter user lable**, and provide the name for this pin (e.g. **LED_RED**). This step helps programming afterward more memorable.

Finally, save the configuration process by pressing **Ctrl + S** and confirm this step by clicking on **OK** button. The code generation is started.

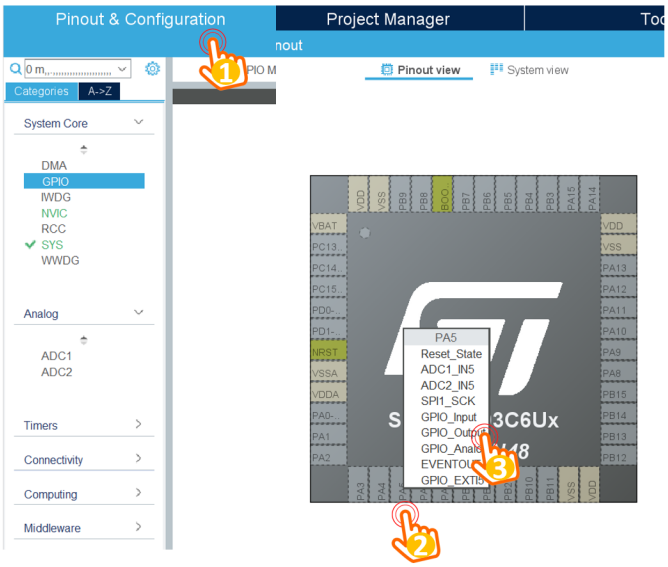
Step 7: Implement the first blinky project in the main function as follow:

```
int main(void)
{
  /* USER CODE BEGIN 1 */

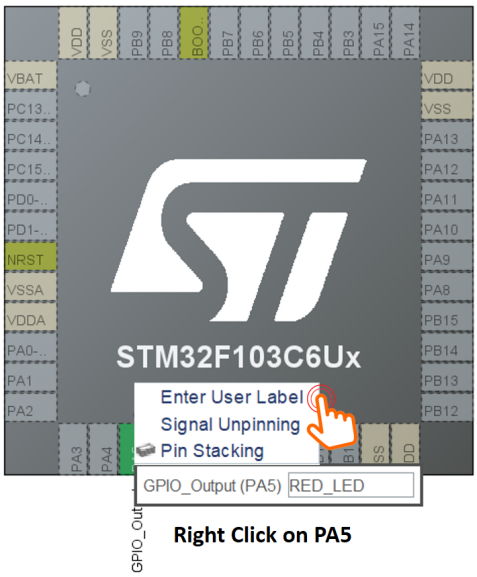
  /* USER CODE END 1 */
```



Hình 1.5: Keep default firmware version



Hình 1.6: Set PA5 to GPIO Output mode



Hình 1.7: Provide a name for PA5

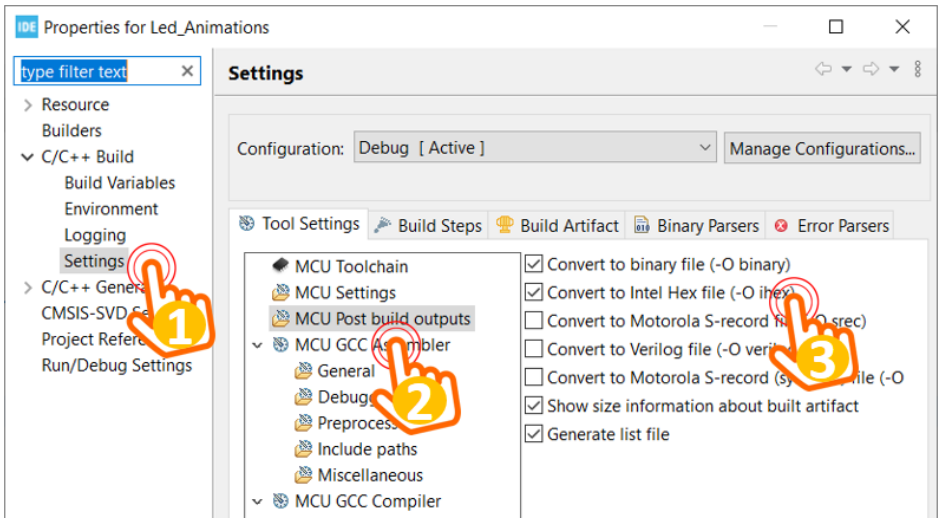
```
6
7   /* MCU Configuration
     ---------------------------------------------------------
     */
8
9   /* Reset of all peripherals, Initializes the Flash
     interface and the Systick. */
10  HAL_Init();
11
12  /* USER CODE BEGIN Init */
13
14  /* USER CODE END Init */
15
16  /* Configure the system clock */
17  SystemClock_Config();
18
19  /* USER CODE BEGIN SysInit */
20
21  /* USER CODE END SysInit */
22
23  /* Initialize all configured peripherals */
24  MX_GPIO_Init();
25  /* USER CODE BEGIN 2 */
26
27  /* USER CODE END 2 */
28
29  /* Infinite loop */
30  /* USER CODE BEGIN WHILE */
31
32  while (1)
33  {
```

```
34      HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
35      HAL_Delay(1000);
36      /* USER CODE END WHILE */
37
38      /* USER CODE BEGIN 3 */
39    }
40    /* USER CODE END 3 */
41  }
```

Program 1.1: First blinky LED project

Actually, what is added to the main function is line number 34 and 35. Please put your code in a right place, otherwise it can be deleted when the code is generated (e.g. change the configuration of the project). When coding, frequently use the suggestions by pressing **Ctrl+Space**.

Step 7: Due to the simulation on Proteus, the hex file should be generated from STM32Cube IDE. From menu **Project**, select **Properties** to open the dialog bellow:



Hình 1.8: Config for hex file output

Navigate to **C/C++ Build**, select **Settings, MCU Post build outputs**, and check to the **Intel Hex file**.

Step 8: Build the project by clicking on menu **Project** and select **Build Project**. Please check on the output console of the IDE to be sure that the hex file is generated, as follow:

```
22:36:06 **** Incremental Build of configuration Debug for project Led_Animations ****
make -j8 all
arm-none-eabi-size   Led_Animations.elf
   text    data     bss     dec     hex filename
   4596      20    1572    6188    182c Led_Animations.elf
Finished building: default.size.stdout


22:36:06 Build Finished. 0 errors, 0 warnings. (took 272ms)
```

Hình 1.9: Compile the project and generate Hex file

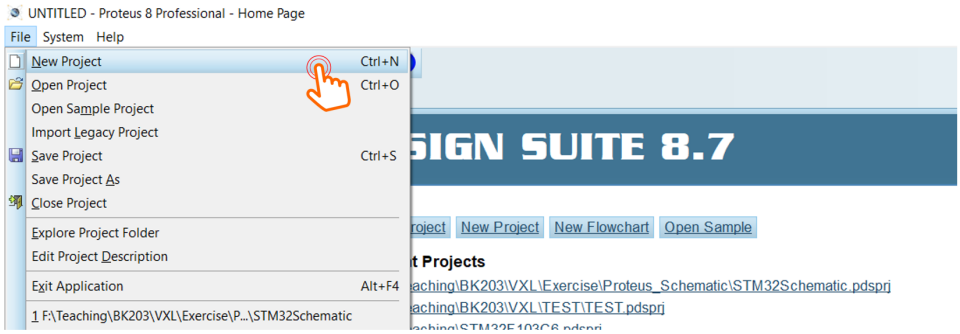
The hex file is located under the **Debug** folder of your project, which is used for the simulation in Proteus afterward. In the case a development kit is connected to your PC, from menu **Run**, select **Run** to download the program to the hardware platform.

In the case there are multiple project in a work-space, double click on the project name to activate this project. Whenever a project is built, check the output files to make sure that you are working in a right project.

3 Simulation on Proteus

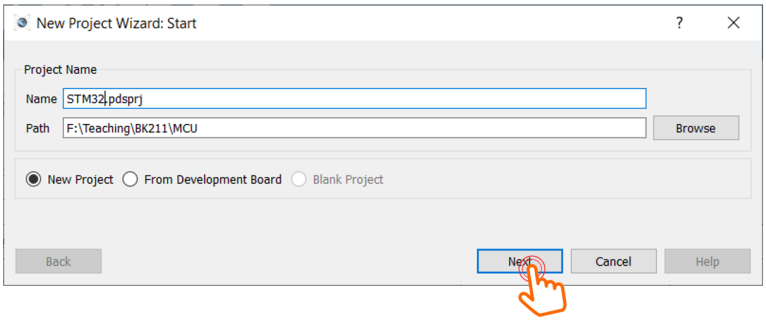
For an online training, a simulation on Proteus can be used. The details to create an STM32 project on Proteus are described bellow.

Step 1: Launch Proteus (**with administration access**) and from menu **File**, select **New Project**.



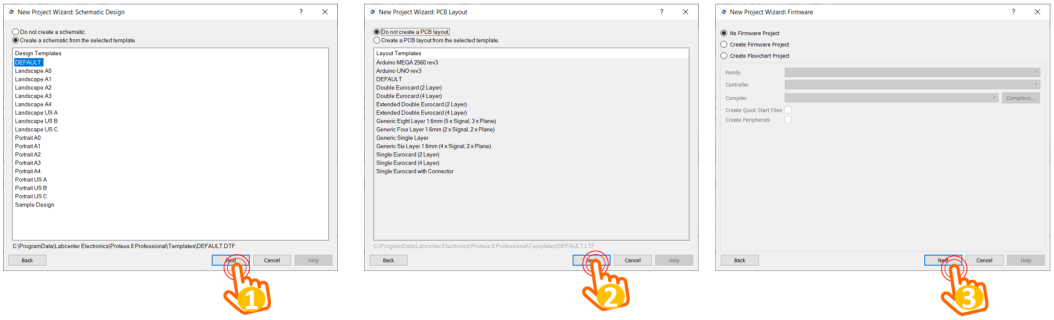
Hình 1.10: Create a new project on Proteus

Step 2: Provide the name and the location of the project, then click on **Next** button.



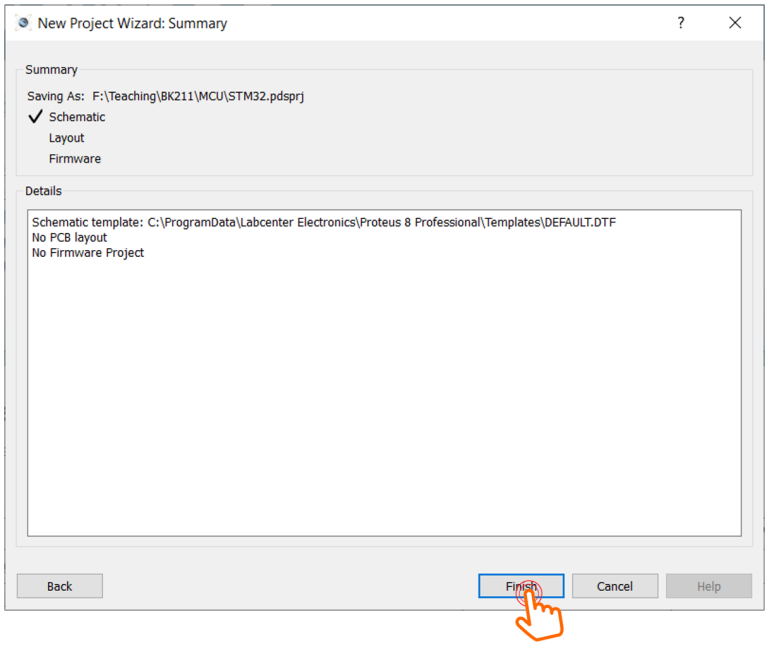
Hình 1.11: Provide project name and location

Step 3: For following dialog, just click on **Next** button as just a schematic is required for the lab.



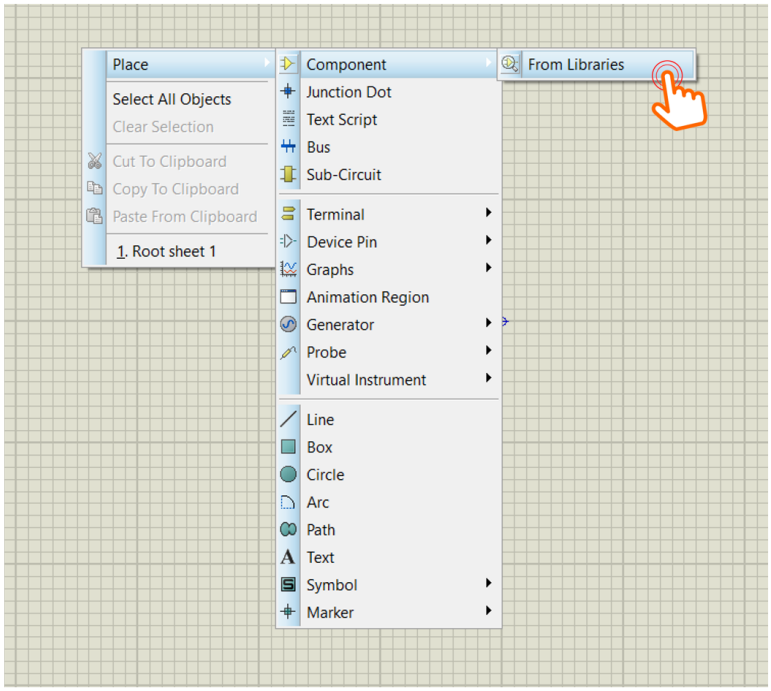
Hình 1.12: Keep the default options by clicking on Next

Step 4: Finally, click on **Finish** button to close the project wizard.



Hình 1.13: Finish the project wizard

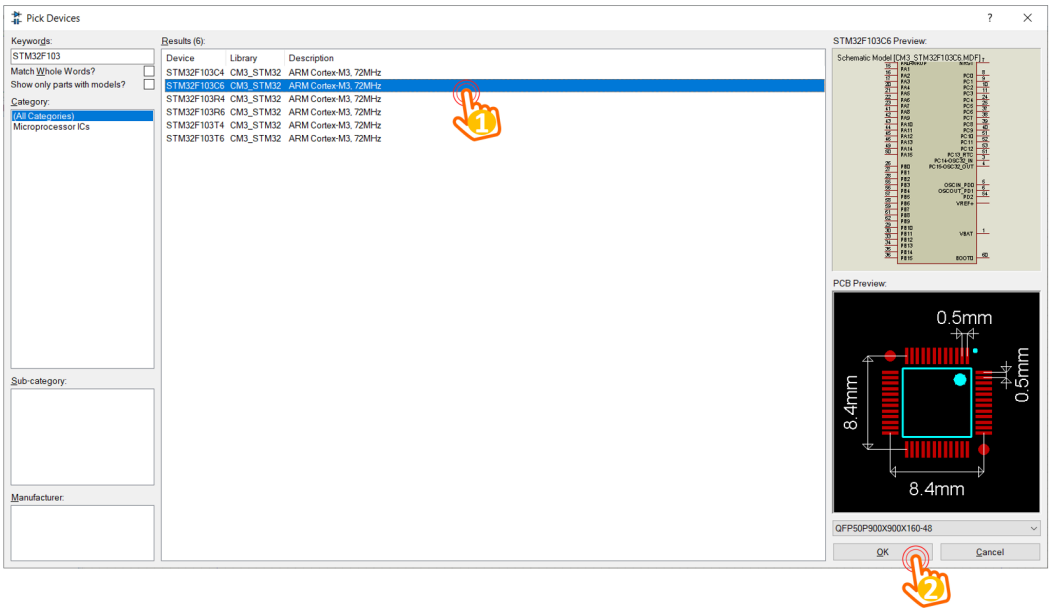
Step 5: On the main page of the project, right click to select **Place, Components, From Libraries**, as follows:



Hình 1.14: Select a component from the library

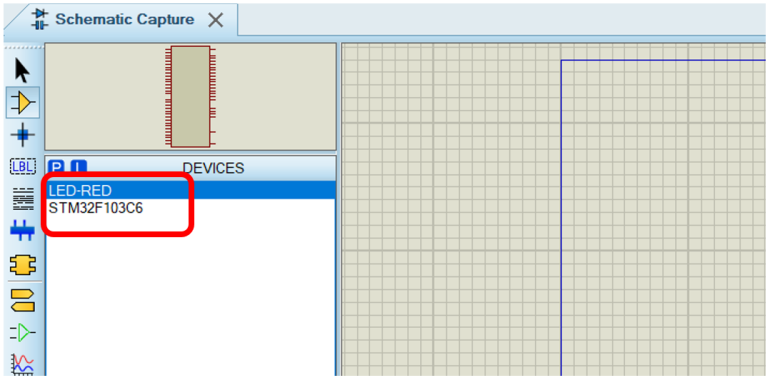
If there is an error with no library found, please restart the Proteus software with Run as administrator option.

Step 6: From the list of components in the library, select STM32F103C6, as follows:



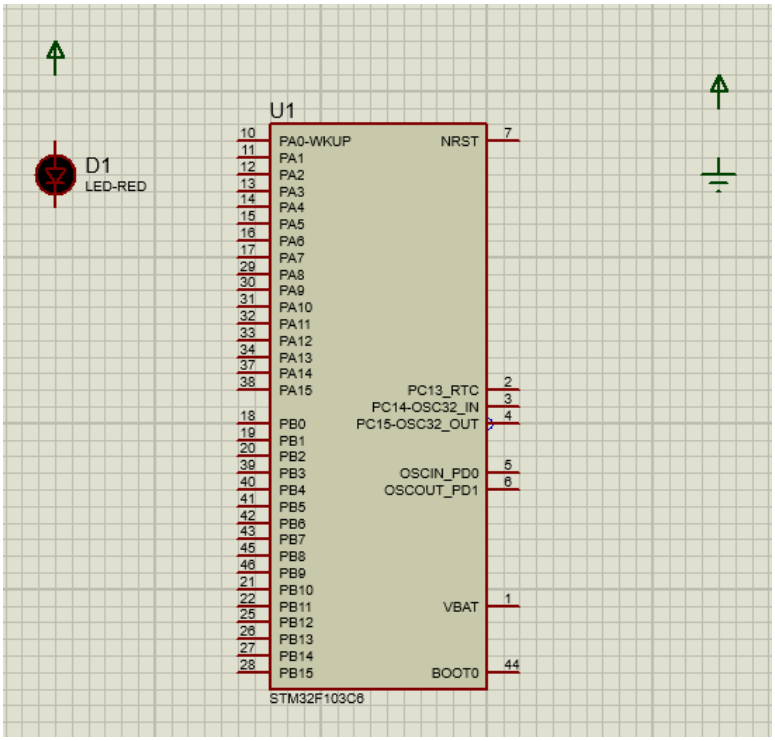
Hình 1.15: Select STM32F103C6

Repeat step 5 and 6 to select an LED, named **LED-RED** in Proteus. Finally, these components are appeared on the DEVICES windows, which is on left hand side as follows:



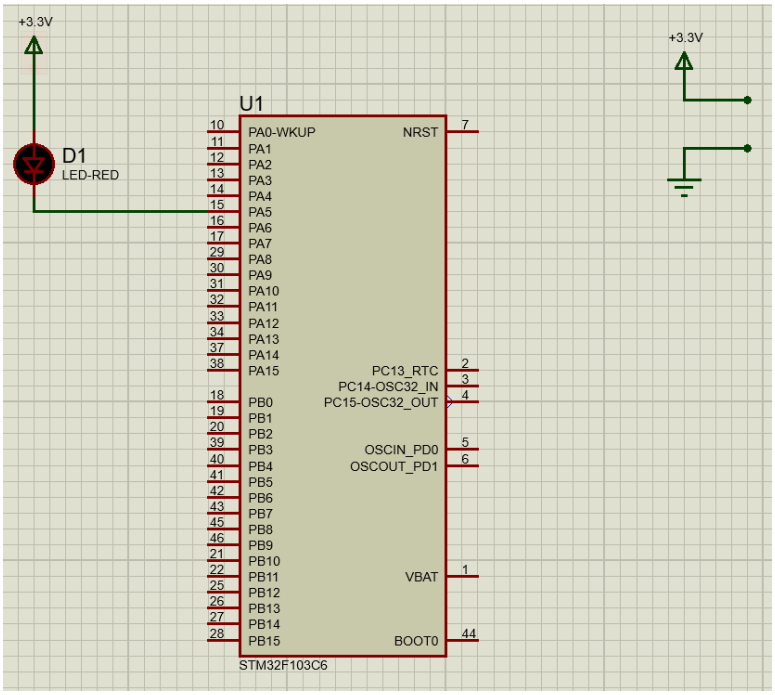
Hình 1.16: STM32 and an LED in the project

Step 7: Place the components to the project: right click on the main page, select on **Place, Component**, and select device added in Step 6. To add the Power and the Ground, right click on the main page, select on **Place, Terminal**. The result in this step is expected as follows:



Hình 1.17: *Place components to the project*

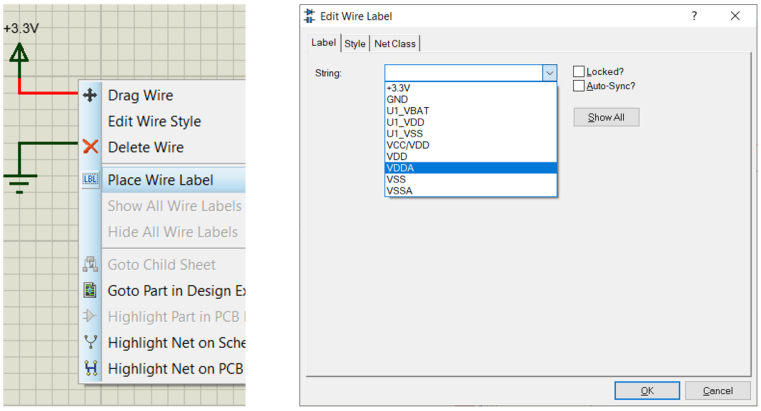
Step 8: Start wiring the circuit. The negative pin of the LED is connected to PA5 while its positive pin is connected to the power supply. For the power and the ground on the right, just make a short wire, which will labeled in the next step.



Hình 1.18: *Connect components and set the power to 3.3V*

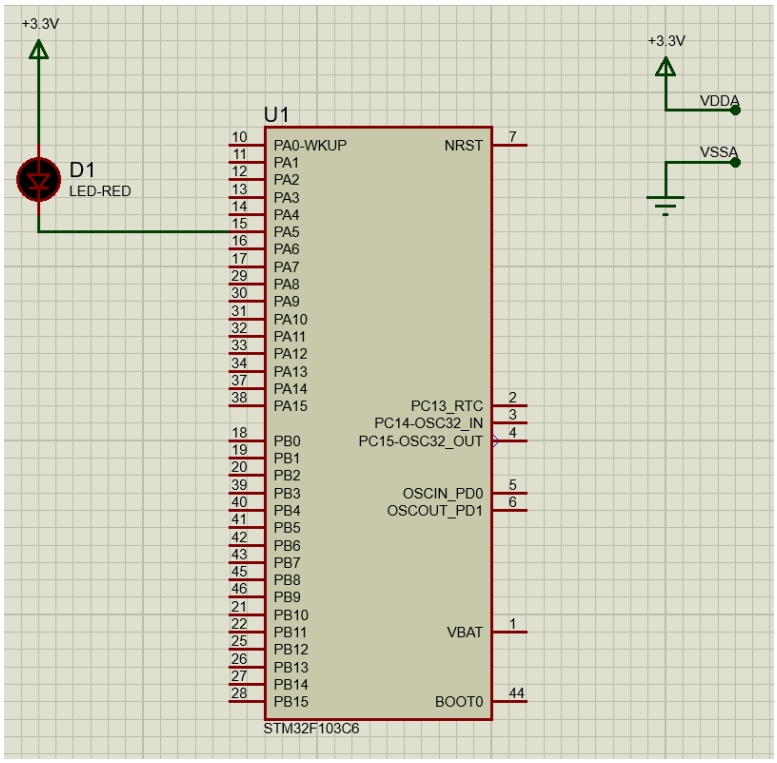
In this step, also double click on the power supply in order to provide the String property to **+3.3V**.

Step 8: Right click on the wire of the power supply and the ground, and select **Place wire Label**



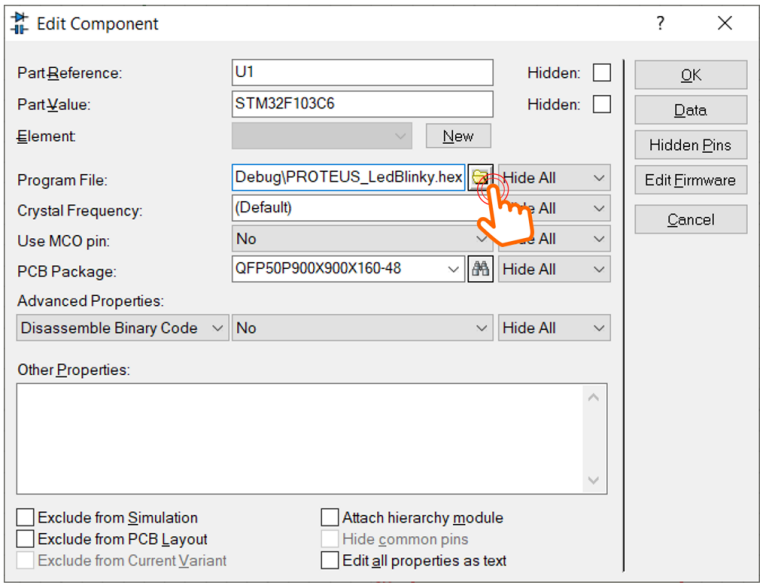
Hình 1.19: *Place label for Power and Ground*

This step is required as VDDA and VSSA of the STM32 must be connected to provide the reference voltage. Therefore, VDDA is connected to 3.3V, while the VSSA is connected to the Ground. Finally, the image of our schematic is shown bellow:



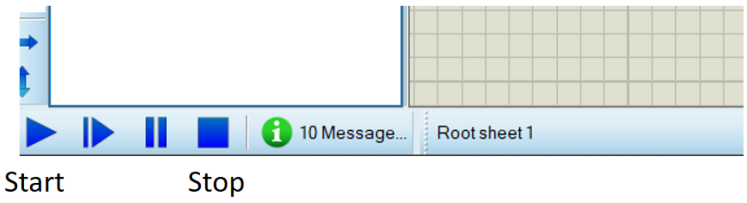
Hình 1.20: *Finalize the schematic*

Step 9: Double click on the STM32, and set the **Program File** to the Hex file, which is generated from Cube IDE, as following:



Hình 1.21: *Set the program of the STM32 to the hex file from Cube IDE*

From now, the simulation is ready to start by clicking on the menu **Debug**, and select on **Run simulation**. To stop the simulation, click on **Debug** and select **Stop VMS Debugging**. Moreover, there are some quick access bottom on the left corner of the Proteus to start or stop the simulation, as shown following:



Hình 1.22: *Quick access buttons to start and stop the simulation*

If everything is success, students can see the LED is blinking every second. Please stop the simulation before updating the project, either in Proteus or STM32Cube IDE. However, the step 9 (set the program file for STM32 in Proteus) is required to do once. Beside the toggle instruction, student can set or reset a pin as following:

```
while (1){
  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
   GPIO_PIN_SET);
  HAL_Delay(1000);
  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
   GPIO_PIN_RESET);
  HAL_Delay(1000);
}
```

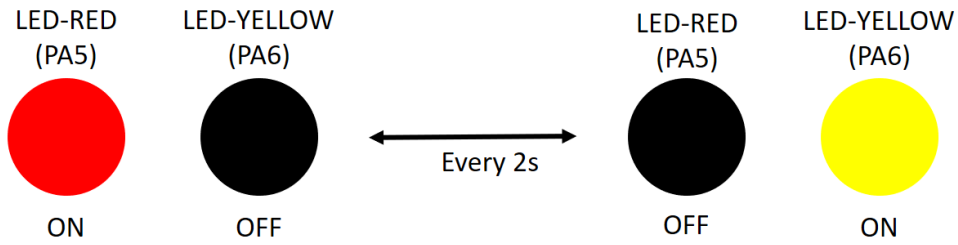
Program 1.2: An example for LED blinky

4 Exercise and Report

4.1 Exercise 1

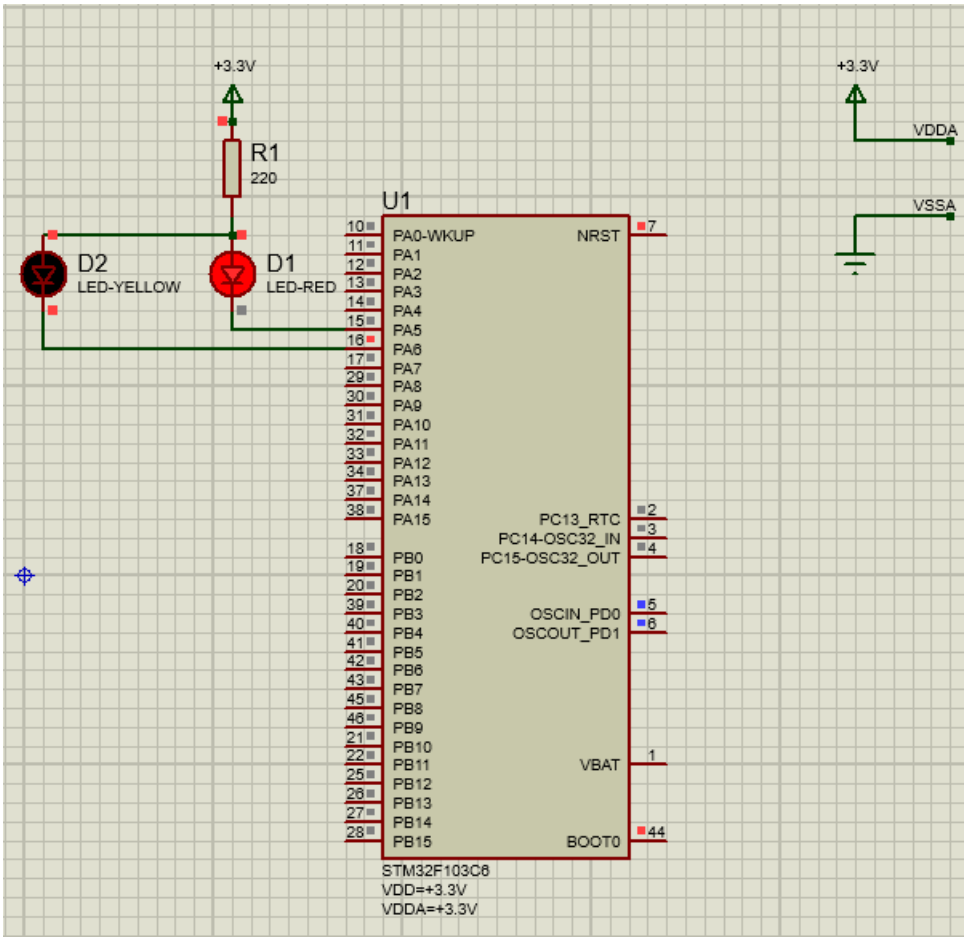
From the simulation on Proteus, one more LED is connected to pin **PA6** of the STM32 (negative pin of the LED is connected to PA6). The component suggested in this exercise is **LED-YELLOW**, which can be found from the device list.

In this exercise, the status of two LEDs are switched every 2 seconds, as demonstrated in the figure bellow.

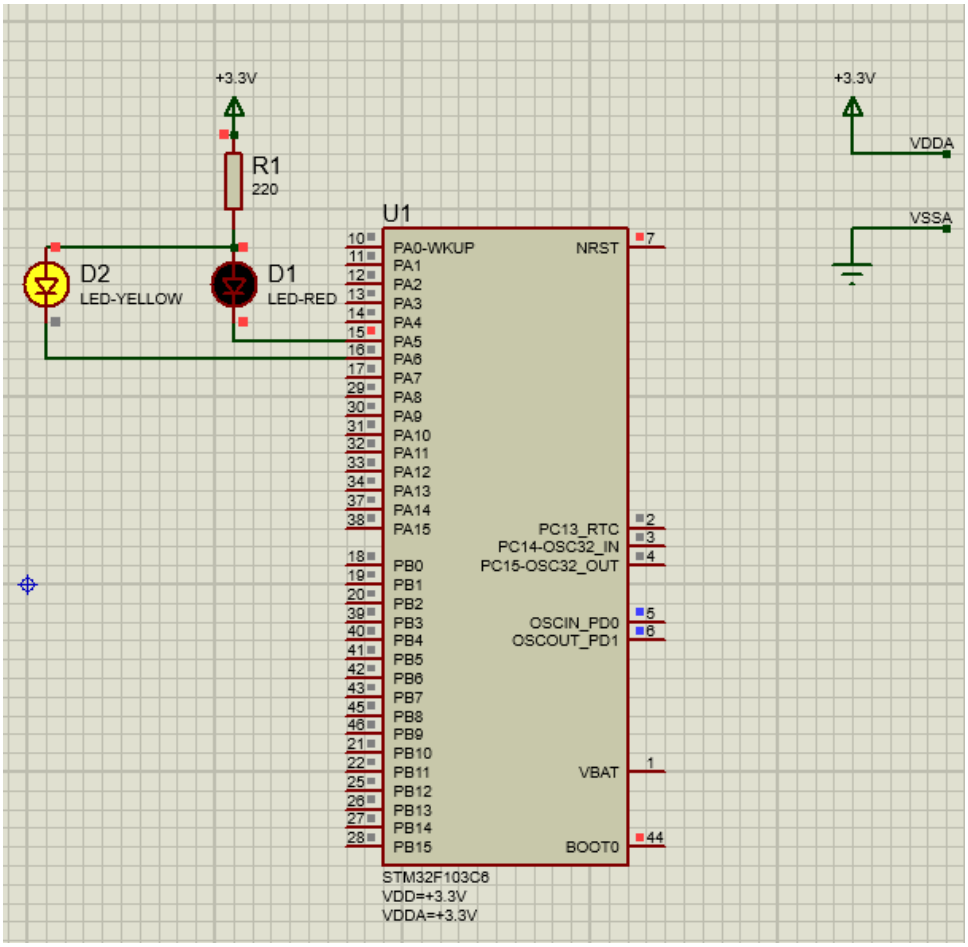


Hình 1.23: State transitions for 2 LEDs

Report 1: Depict the schematic from Proteus simulation in this report. The caption of the figure is a downloadable link to the Proteus project file (e.g. a github link).



Hình 1.24: When the RED LED On, YEL LED Off *EX1_LAB1*



Hình 1.25: When the RED LED Off, YEL LED On *EX1_LAB1*

Report 2: Present the source code in the infinite loop while of your project. If a user-defined functions is used, it is required to present in this part. A brief description can be added for this function (e.g. using comments). A template to present your source code is presented bellow.

```
  while (1)
  {
#if LAB1_EXERCISE == 1U
    runExercise1Cycle();
#elif LAB1_EXERCISE == 2U
    runExercise2Cycle();
#else
#error "LAB1_EXERCISE must be 1 or 2"
#endif


    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
```

Program 1.3: while(1) loop

```
/* USER CODE BEGIN 0 */
static void allLedsOff(void)
{
  /* The LEDs in this Proteus circuit are active-low. */
  HAL_GPIO_WritePin(GPIOA, LED_RED_Pin | LED_YELLOW_Pin |
   LED_GREEN_Pin,
                    GPIO_PIN_SET);
}

static void __attribute__((unused)) runExercise1Cycle(void)
{
  /* Exercise 1: red and yellow alternate every 2 seconds.
   */
  allLedsOff();
  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
   GPIO_PIN_RESET);
  HAL_Delay(2000);

  allLedsOff();
  HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
   GPIO_PIN_RESET);
  HAL_Delay(2000);
}
```

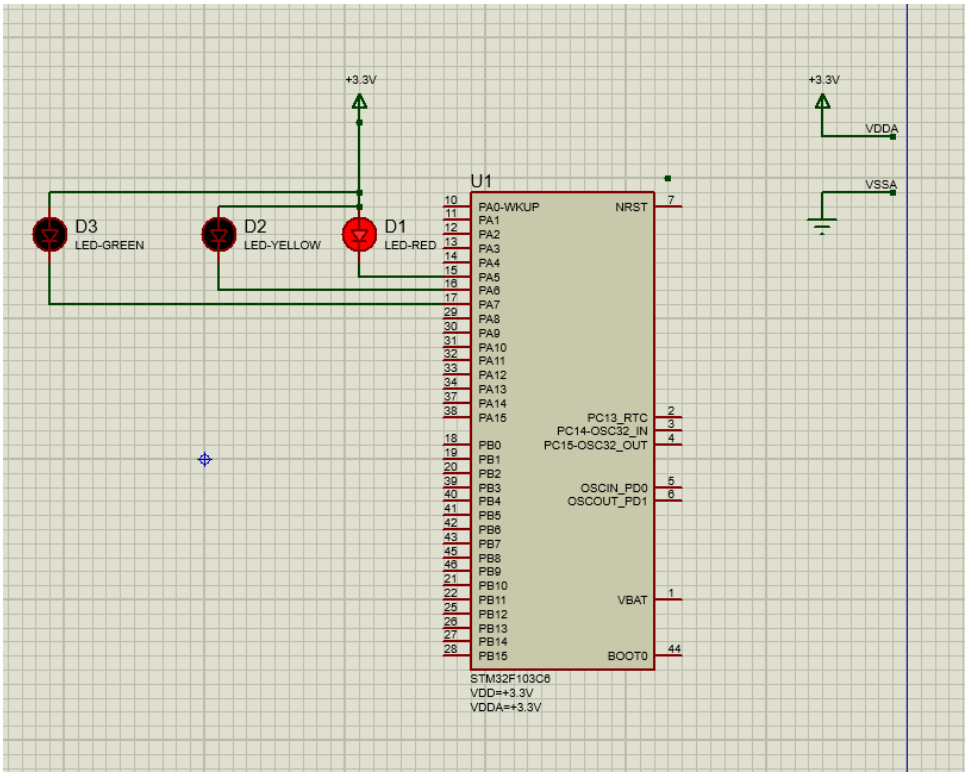
Program 1.4: allLedsOff() and runExercise1Cycle() Functions

4.2 Exercise 2

Extend the first exercise to simulate the behavior of a traffic light. A third LED, named **LED-GREEN** is added to the system, which is connected to **PA7**. A cycle in this traffic light is 5 seconds for the RED, 2 seconds for the YELLOW and 3 seconds for the GREEN. The LED-GREEN is also controlled by its negative pin.

Similarly, the report in this exercise includes the schematic of your circuit and a your source code in the while loop.

Report 1: Present the schematic.



Hình 1.26: Simulation For Ex2 EX2 Github

Report 2: Present the source code in while.

```
static void __attribute__((unused)) runExercise2Cycle(void)
{
  /* Exercise 2: red 5 s, yellow 2 s, green 3 s. */
  allLedsOff();
  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
   GPIO_PIN_RESET);
  HAL_Delay(5000);

  allLedsOff();
  HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
   GPIO_PIN_RESET);
  HAL_Delay(2000);

  allLedsOff();
  HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
   GPIO_PIN_RESET);
  HAL_Delay(3000);
}
/* USER CODE END 0 */
```

Program 1.5: runExercise2Cycle() function

```
  while (1)
  {
#if LAB1_EXERCISE == 1U
    runExercise1Cycle();
#elif LAB1_EXERCISE == 2U
    runExercise2Cycle();
#else
#error "LAB1_EXERCISE must be 1 or 2"
#endif

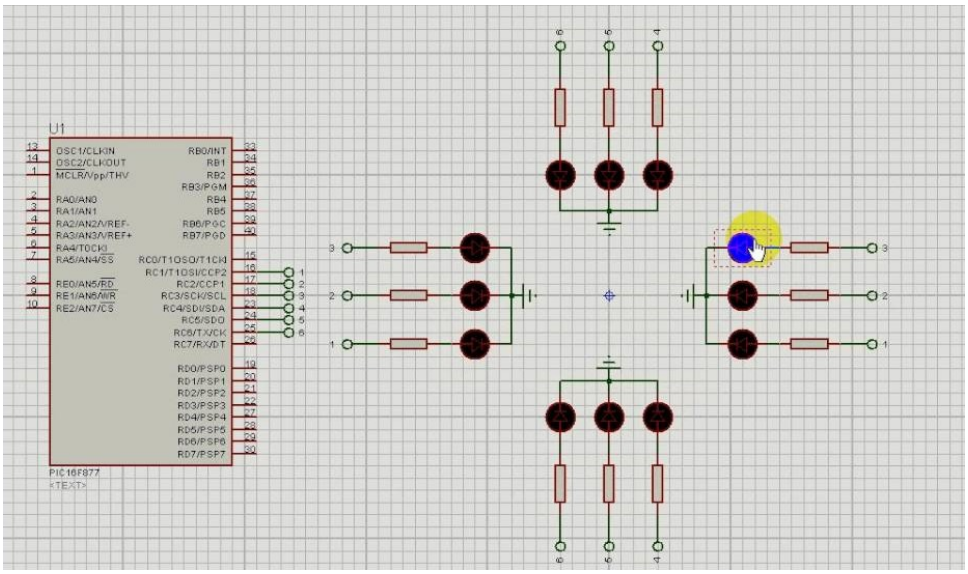

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
```

Program 1.6: in while (1) loop

4.3 Exercise 3

Extend to the 4-way traffic light. Arrange 12 LEDs in a nice shape to simulate the behaviors of a traffic light. A reference design can be found in the figure bellow.

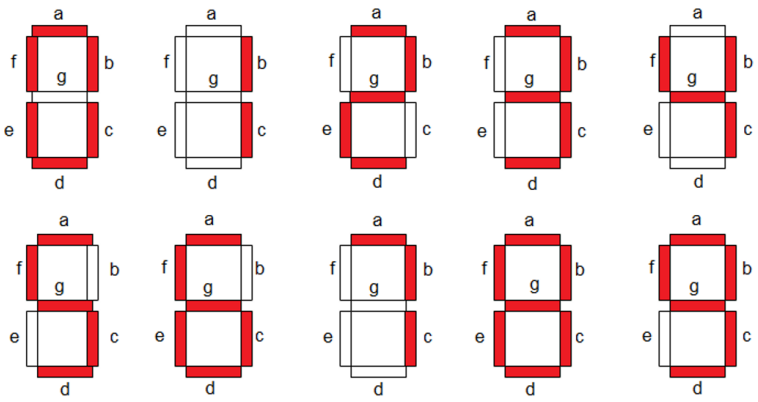


Hình 1.27: *Reference design for a 4 way traffic light*

4.4 Exercise 4

Add **only one 7 led segment** to the schematic in Exercise 3. This component can be found in Proteus by the keyword **7SEG-COM-ANODE**. For this device, the common pin should be connected to the power supply and other pins are supposed to connected to PB0 to PB6. Therefore, to turn-on a segment in this 7SEG, the STM32 pin should be in logic 0 (0V).

Implement a function named **display7SEG(int num)**. The input for this function is from 0 to 9 and the outputs are listed as following:



Hình 1.28: *Display a number on 7 segment LED*

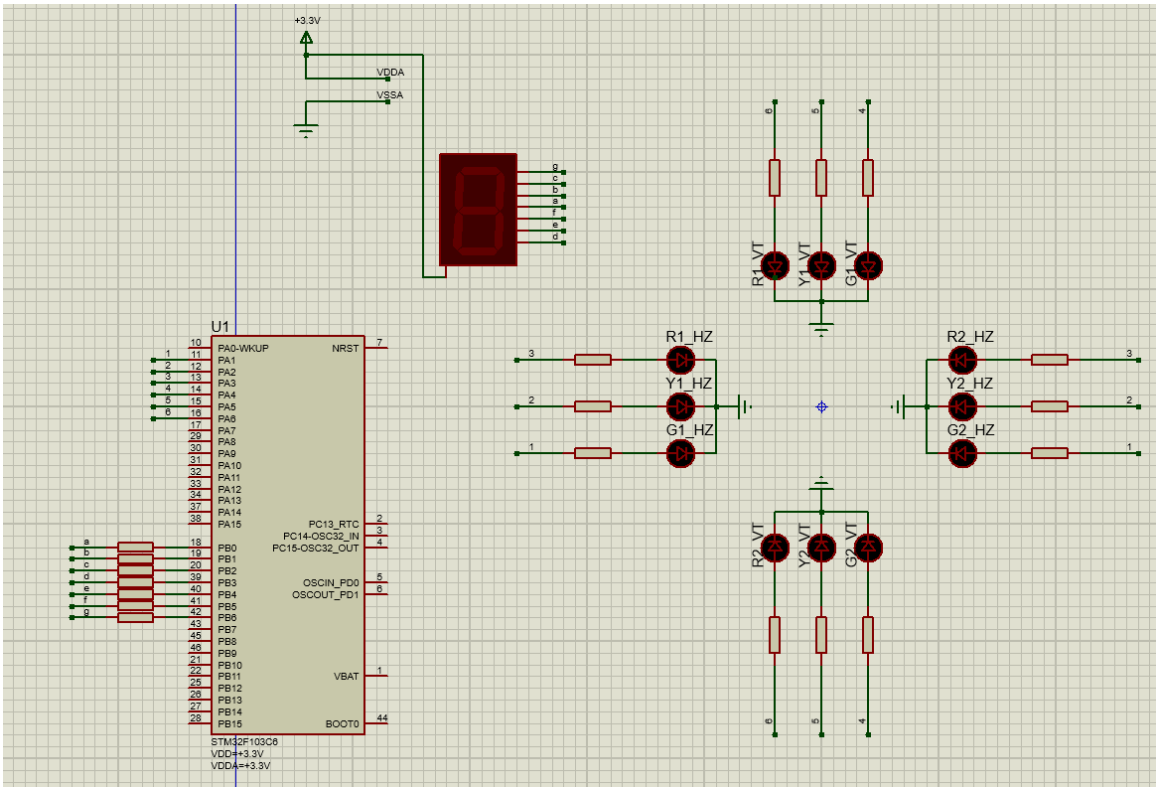
This function is invoked in the while loop for testing as following:

```
int counter = 0;
while (1){
    if(counter >= 10) counter = 0;
    display7SEG(counter++);
    HAL_Delay(1000);

}
```

Program 1.7: An example for your source code

Report 1: Present the schematic.



Hình 1.29: Schematic for Exercise 345. Github: Github Ex345

The schematic includes a 7-segment display showing numbers from 0 to 9, representing time in seconds. There are two sets of traffic lights (green, yellow, and red); "Hz" designates the horizontal row of lights, while "Vt" designates the vertical row.

Report 2: Present the source code for display7SEG function.

```
//bit0=a, bit1=b, ... bit6=g
static const uint8_t segCode[10] = {
      /*0*/ 0b1000000,  // abcdef ON, g OFF
      /*1*/ 0b1111001,  // b c
      /*2*/ 0b0100100,  // a b d e g
      /*3*/ 0b0110000,  // a b c d g
      /*4*/ 0b0011001,  // b c f g
      /*5*/ 0b0010010,  // a c d f g
      /*6*/ 0b0000010,  // a c d e f g
      /*7*/ 0b1111000,  // a b c
      /*8*/ 0b0000000,  // all
      /*9*/ 0b0010000   // a b c d f g
};
// Mapping GPIO Pins a..g to PBx pins in Proteus.
// a->PB0, b->PB1, ..., g->PB6 in CubeIDE GPIO Pins
   Configure
static const uint16_t segPins[7] = {
  GPIO_PIN_0, // a -> PB0
  GPIO_PIN_1, // b -> PB1
  GPIO_PIN_2, // c -> PB2
  GPIO_PIN_3, // d -> PB3
  GPIO_PIN_4, // e -> PB4
  GPIO_PIN_5, // f -> PB5
  GPIO_PIN_6  // g -> PB6
};

// 0-9 numbers on 7-seg COM-ANODE (0=ON, 1=OFF)
void display7SEG(uint8_t num)
{
  if (num > 9) return;
  uint8_t code = segCode[num];

  for (int i = 0; i < 7; ++i) {
    /* bit = 1 (OFF) -> SET (HIGH); bit = 0 (ON) -> RESET (
   LOW) */
    GPIO_PinState st = (code & (1U << i)) ? GPIO_PIN_SET :
   GPIO_PIN_RESET;
    HAL_GPIO_WritePin(GPIOB, segPins[i], st);
  }
}
```

Program 1.8: display7SEG function source code

4.5 Exercise 5

Integrate the 7SEG-LED to the 4 way traffic light. In this case, the 7SEG-LED is used to display countdown value.

In this exercise, only source code is required to present. The function display7SEG in previous exercise can be re-used.

```
static void test7SEG(void)
{
  /* Exercise 4: show every digit once before the traffic cycle starts. */
  for (uint8_t digit = 0; digit < 10U; ++digit) {
    display7SEG(digit);
    HAL_Delay(1000);
  }
}

static void setTrafficLights(uint16_t horizontalPin, uint16_t verticalPin)
{
  const uint16_t horizontalPins = Hz_Green_Pin | Hz_Yellow_Pin | Hz_Red_Pin;
  const uint16_t verticalPins = Vt_Green_Pin | Vt_Yellow_Pin | Vt_Red_Pin;

  /* Turn everything off first, then enable exactly one light per direction. */
  HAL_GPIO_WritePin(GPIOA, horizontalPins | verticalPins, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOA, horizontalPin | verticalPin, GPIO_PIN_SET);
}

static void countdown(uint8_t seconds)
{
  for (uint8_t value = seconds; value > 0U; --value) {
    display7SEG(value);
    HAL_Delay(1000);
  }
}

//in while(1) loop
  while (1)
  {
    /* Exercises 3 and 5: safe two-direction traffic-light cycle. */
    setTrafficLights(Hz_Green_Pin, Vt_Red_Pin);
    countdown(3);
```

```
34
35      setTrafficLights(Hz_Yellow_Pin, Vt_Red_Pin);
36      countdown(2);
37
38      setTrafficLights(Hz_Red_Pin, Vt_Green_Pin);
39      countdown(3);
40
41      setTrafficLights(Hz_Red_Pin, Vt_Yellow_Pin);
42      countdown(2);
43
44
45      /* USER CODE END WHILE */
46
47      /* USER CODE BEGIN 3 */
48  }
```

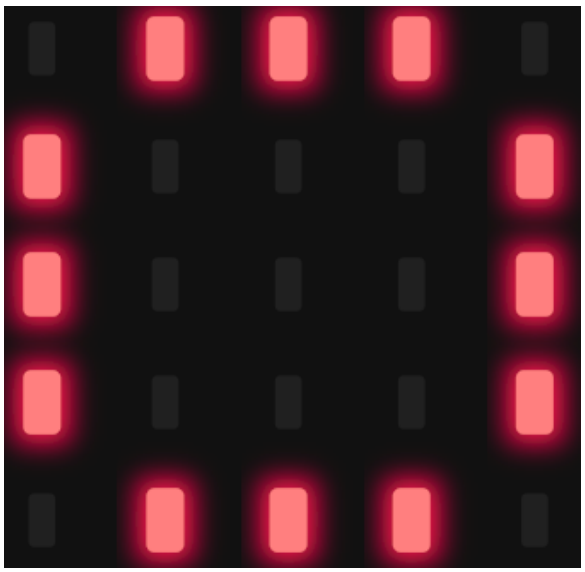
Program 1.9: EX5 source code

The sequence is as follows: the horizontal traffic lights turns on green LED and the vertical turns on red LED for 3 seconds; the horizontal turns on yellow LED and the vertical turns on red LED for 2 seconds; the horizontal turns on red LED and the vertical turns on green LED for 3 seconds; the horizontal turns on red LED and the vertical turns on yellow LED for 3 seconds; then the cycle repeats.

Ex345 source code Github Link

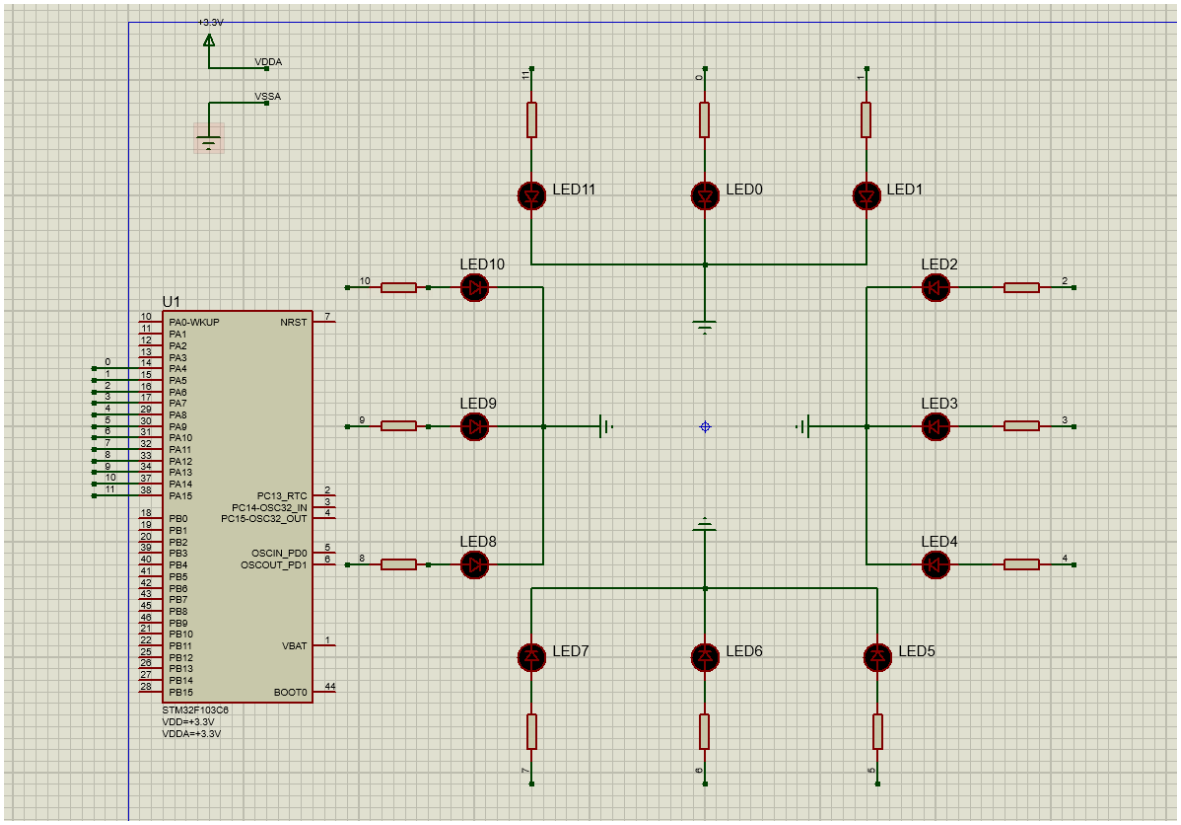
4.6 Exercise 6

In this exercise, a new Proteus schematic is designed to simulate an analog clock, with 12 different number. The connections for 12 LEDs are supposed from PA4 to PA15 of the STM32. The arrangement of 12 LEDs is depicted as follows.



Hình 1.30: *12 LEDs for an analog clock*

Report 1: Present the schematic.



Hình 1.31: Ex6 Simulation

Assigned sequence numbers 0–11 to the LEDs, corresponding to pins PA4–15.

Report 2: Implement a simple program to test the connection of every single LED. This testing program should turn every LED in a sequence.

```
#define CLOCK_ALL_PINS 0xFFF0U

static void clearAllClock(void)
{
    HAL_GPIO_WritePin(
        GPIOA,
        CLOCK_ALL_PINS,
        GPIO_PIN_RESET
    );
}

static void turnOnClockLed(uint8_t number)
{
    if (number > 11U) {
        return;
    }

    uint16_t pin = (uint16_t)(GPIO_PIN_4 << number);

    HAL_GPIO_WritePin(
        GPIOA,
        pin,
        GPIO_PIN_SET
    );
}

// LED GPIO lists (PA4 to PA15)
const uint16_t LED_Pins[12] = {
    GPIO_PIN_4, GPIO_PIN_5, GPIO_PIN_6, GPIO_PIN_7,
    GPIO_PIN_8, GPIO_PIN_9, GPIO_PIN_10, GPIO_PIN_11,
    GPIO_PIN_12, GPIO_PIN_13, GPIO_PIN_14, GPIO_PIN_15
};

static void LED_Test_Sequence(void)
{
    for (uint8_t number = 0U; number < 12U; ++number) {
        clearAllClock();
        turnOnClockLed(number);
        HAL_Delay(500);
    }

    clearAllClock();
}
// In while(1) loop
```

```
45   while (1)
46   {
47     LED_Test_Sequence(); // Check each of the 12 LEDs.
48   }
```

Program 1.10: Ex6 Source Code

EX6_LAB1 source code

4.7 Exercise 7

Implement a function named **clearAllClock()** to turn off all 12 LEDs. Present the source code of this function.

```
1  static void clearAllClock(void)
2  {
3      HAL_GPIO_WritePin(
4          GPIOA,
5          CLOCK_ALL_PINS,
6          GPIO_PIN_RESET
7      );
8  }
9
10 //in while(1) loop
11 while (1)
12 {
13     // Turn ON all LEDs
14     HAL_GPIO_WritePin(
15         GPIOA,
16         CLOCK_ALL_PINS,
17         GPIO_PIN_SET
18     );
19
20     HAL_Delay(1000);
21
22     // Turn OFF all LEDs
23     clearAllClock();
24
25     HAL_Delay(1000);
26 }
```

Program 1.11: Ex7 source code

Result:

12 LEDs light up simultaneously for 1 second → turn off simultaneously for 1 second → repeat

4.8 Exercise 8

Implement a function named **setNumberOnClock(int num)**. The input for this function is from **0 to 11** and an appropriate LED is turn on. Present the source code of this function.

```
void setNumberOnClock(uint8_t number)
{
    if (number > 11U) {
        return;
    }

    uint16_t pin =
        (uint16_t)(GPIO_PIN_4 << number);

    HAL_GPIO_WritePin(
        GPIOA,
        pin,
        GPIO_PIN_SET
    );
}

// in while(1) loop

while (1)
{
    for (uint8_t number = 0U; number < 12U; ++number) {
        clearAllClock();
        setNumberOnClock(number);
        HAL_Delay(500);
    }
}
```

Program 1.12: Ex8 source code

Result:

LED0 → LED1 → LED2 → ... → LED11

4.9 Exercise 9

Implement a function named **clearNumberOnClock(int num)**. The input for this function is from **0 to 11** and an appropriate LED is turn off.

```
void clearNumberOnClock(uint8_t number)
{
    if (number > 11U) {
        return;
    }

    uint16_t pin =
        (uint16_t)(GPIO_PIN_4 << number);

    HAL_GPIO_WritePin(
        GPIOA,
        pin,
        GPIO_PIN_RESET
    );
}

// In while(1) loop
while (1)
{
    HAL_GPIO_WritePin(
        GPIOA,
        CLOCK_ALL_PINS,
        GPIO_PIN_SET
    );

    HAL_Delay(1000);

    for (uint8_t number = 0U; number < 12U; ++number) {
        clearNumberOnClock(number);
        HAL_Delay(500);
    }

    HAL_Delay(1000);
}
```

Program 1.13: Ex9 source code

Result:

All LEDs on → LED0 off → LED1 off → ... → LED11 off → repeat

4.10 Exercise 10

Integrate the whole system and use 12 LEDs to display a clock. At a given time, there are only 3 LEDs are turn on for hour, minute and second information.

The three values are converted to positions 0–11 on the clock face:

Hour hand: hour Minute hand: minute / 5. Second hand: second / 5.

```
#define CLOCK_ALL_PINS 0xFFF0U

void clearAllClock(void)
{
    HAL_GPIO_WritePin(
        GPIOA,
        CLOCK_ALL_PINS,
        GPIO_PIN_RESET
    );
}

void setNumberOnClock(uint8_t number)
{
    if (number > 11U) {
        return;
    }

    uint16_t pin =
        (uint16_t)(GPIO_PIN_4 << number);

    HAL_GPIO_WritePin(
        GPIOA,
        pin,
        GPIO_PIN_SET
    );
}

void clearNumberOnClock(uint8_t number)
{
    if (number > 11U) {
        return;
    }

    uint16_t pin =
        (uint16_t)(GPIO_PIN_4 << number);

    HAL_GPIO_WritePin(
        GPIOA,
        pin,
        GPIO_PIN_RESET
    );
}
```

```
43
44 void ShowClock(
45     uint8_t hour,
46     uint8_t minute,
47     uint8_t second
48 )
49 {
50     clearAllClock();
51
52     setNumberOnClock(hour % 12U);
53     setNumberOnClock(minute / 5U);
54     setNumberOnClock(second / 5U);
55 }
```

Program 1.14: A full clock program

A clock cycle consists of 60 minutes but has only 12 LEDs:

60 / 12 = 5

Therefore, each LED represents 5 minutes or 5 seconds:

Bảng 1.1:

Minutes/Seconds	**Division**	**LED**
0–4	/5	LED0
5–9	/5	LED1
10–14	/5	LED2
15–19	/5	LED3
20–24	/5	LED4
25–29	/5	LED5
30–34	/5	LED6
35–39	/5	LED7
40–44	/5	LED8
45–49	/5	LED9
50–54	/5	LED10
55–59	/5	LED11

Example:

ShowClock(10, 30, 45);

Result:

Hour: 10 % 12 = 10 → LED10 lights up Minute: 30 / 5 = 6 → LED6 lights up Second: 45 / 5 = 9 → LED9 lights up

LEDs 10, 6, and 9 will all light up simultaneously.

Case where the hands overlap

Example:

ShowClock(3, 15, 16);

We have:

Hour: 3 % 12 = 3 Minute: 15 / 5 = 3 Second: 16 / 5 = 3

All three values point to LED3, so only one LED appears lit on the circuit. This is normal behavior because all three "hands" share the same LED system.

Declare the start time:

```
uint8_t hour = 10U;
uint8_t minute = 58U;
uint8_t second = 50U;
```

Program 1.15: Ex10 source code

The time will run:

10:58:50 10:58:51 ... 10:58:59 10:59:00 ... 10:59:59 11:00:00

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    /* USER CODE BEGIN 2 */

    uint8_t hour = 10U;
    uint8_t minute = 58U;
    uint8_t second = 50U;

    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */

    while (1)
    {
        ShowClock(hour, minute, second);
        HAL_Delay(1000);

        second++;

        if (second >= 60U) {
            second = 0U;
            minute++;
        }

        if (minute >= 60U) {
            minute = 0U;
            hour++;
        }

        if (hour >= 24U) {
            hour = 0U;
        }

        /* USER CODE END WHILE */

```

```
41          /* USER CODE BEGIN 3 */
42      }
43
44      /* USER CODE END 3 */
45  }
```

Program 1.16: Ex10 source code

Ex678910 Source Code Github Link